

CSA1419 - Compiler Design

M. Harinath
192427020

C02 - AT2

- ① Define the role of a parser in Compiler Design.
A Parser checks whether the input program follows the grammar of the language and generates a parse tree or syntax tree.
- ② What is a Context-free Grammar (CFG)?
A CFG is a set of production rules used to describe the syntax of a programming language. It contains terminals, non-terminals, a start symbol, and productions.
✓
- ③ Identify Terminals and non-terminals in: $S \rightarrow aSb \mid \epsilon$
 - Non-terminals: S
 - Terminals: a, b
 - ϵ (epsilon) represents the empty string.
- ④ Remove left recursion for: $A \rightarrow Aab$.
Left-recursive grammar.
 - $A \rightarrow bA'$
 - $A' \rightarrow aA' \mid \epsilon$
- ⑤ Apply left factoring for: $S \rightarrow abA \mid abB \mid c$
Left-factored grammar:
 - $S \rightarrow abX \mid c$
 - $X \rightarrow AB$
- ⑥ Find First(S) for: $S \rightarrow aAb$
 $FIRST(S) = \{a, b\}$

⑦ Find $\text{FIRST}(A)$ for: $A \rightarrow aA | \epsilon$
 $\text{FIRST}(A) = \{a, \epsilon\}$

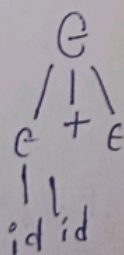
⑧ Find $\text{Follow}(S)$ for: $S \rightarrow AB$
Since S is the start symbol
 $\text{Follow}(S) = \{\$ \}$

⑨ Find $\text{Follow}(B)$ for: $S \rightarrow AB, B \rightarrow b$
 B is at the end of the production, so
 $\text{Follow}(B) = \text{Follow}(S) = \{\$ \}$

⑩ Check whether the grammar is ambiguous: $E \rightarrow E + E | id$.
Yes, the grammar is ambiguous because the string
 $id + id$ has more than one parse tree.

⑪ Construct a parse tree for: $id + id$ using $E \rightarrow E + E | id$.

Parse tree:



⑫ Check whether the string $aaabbb$ is valid for:
 $S \rightarrow aSb | \epsilon$

Yes, the string $aaabbb$ is valid because it can be
derived by repeatedly applying $S \rightarrow aSb$ and
finally $S \rightarrow \epsilon$

13. Check whether the string ab is valid for: $S \rightarrow AA$,
 $A \rightarrow b$.

yes.

Derivation: $S \Rightarrow AA \Rightarrow ab$

14. What is top-down parsing?

Top-down parsing starts from the start symbol and builds the parse tree from the root to the leaves.

15. What is recursive descent parsing?

Recursive descent parsing is a top-down parsing technique in which each non-terminal is implemented as a recursive function.

16. Write the procedure for non-terminal S in: $S \rightarrow aS \mid b$

Procedure $S()$:

if input == 'a'.

match('a')

$S()$

else if input == 'b'

match('b')

else:

error()

17. What is predictive parsing?

Predictive parsing is a top-down parsing method that uses one lookahead symbol and a parsing table to choose productions without backtracking.

(18) State the Condition for a grammar to be LL(1).

A grammar is LL(1) if for every non-terminal, the FIRST sets of alternatives are disjoint, and if ϵ is present, FIRST and FOLLOW sets are disjoint.

(19) What is bottom-up parsing?

Bottom-up parsing starts from the input string and reduces it step by step to the start symbol.

(20) What is shift-reduce parsing?

Shift-reduce parsing is a bottom-up parsing technique that uses a stack and performs shift-reduce operations.

(21) List the four actions in shift-reduce parsing.

The four actions are:

• Shift • Reduce • Accept • Error.

(22) What is operator precedence parsing?

Operator precedence parsing is a bottom-up parsing method that uses precedence relations b/w operators to parse expressions.

(23) What is an LR parser?

An LR parser is a bottom-up parser that reads input from left to right and produces the rightmost derivation in reverse.

(24) What is an SLR parser?

An SLR parser is a bottom-up parser that uses FOLLOW sets to construct the parsing table.

(25) What is panic mode error recovery?

Panic mode error recovery skips input symbols until a synchronizing token is found and then continues parsing.